

- zipped project folder of the exercises 1 and 2
- this document compiled possibly in pdf format.

Exercise 1)

In “Magic the Gathering” a player keeps track of his collection recording for each card the purchase price, card condition (poor, good, mint) and current card market values. Write a program in **ARM assembly** language that analyzes the player collection (**it must work and output the correct result**).

Cards	DCD	0x134, 3, 275, 0x2B9, 0xDC, 151, 2087
Condition	DCD	2087, 2, 275, 0x0, 308, 0x1, 0xDC, 2, 151, 2, 0x3, 0, 697, 2
Purchase price	DCD	0x3, 2000, 0x113, 2, 151, 9, 0x134, 45, 2087, 17, 220, 5, 697, 350
Current price	DCD	0xDC, 3, 151, 16, 3, 3300, 697, 420, 308, 63, 275, 1, 0x827, 3
#Cards	DCB	7

Where:

- Cards is a table where each entry consists of a card identifier.
- Condition is a table where each entry consists of two integer values: the ID of the card (4 bytes) and the condition of the corresponding card:
 - 0: POOR
 - 1: GOOD
 - 2: MINT
- Purchase price is a table where each entry consists of two integer values: the ID of the card (4 bytes) and the value at which the card was originally purchased.
- Current price is a table where each entry consists of two integer values: the ID of the card (4 bytes) and the current market value price.
- #Cards is a 1-byte constant and indicate the number of cards in the player’s collection.

Compute **the number of cards** that the player purchased at a lower price than the current price and store it in register R11. Then for each of these cards compute the difference of current – purchase price, sum it to the other cards, and store it in register R10.

Note: The constant data section must be defined in the code section, with a 2byte alignment and 4096 boundary zero bytes.

Example:

```
...
// ALIGNMENT
// BOUNDARY (SPACE ....)
MY DATA
// BOUNDARY (SPACE ....)
..
```

Exercise 2)

Store the IDs of the cards in three separate vectors, `POOR`, `GOOD`, and `MINT`, based on their condition. Within each vector, sort the IDs in ascending order according to the difference between the current price and the purchase price.

Then, save in `R11` and `R12` the ID and condition of the card that had the highest loss in value.

Compute the needed bytes for the above vectors.

Vector	Size [bytes]
POOR	56
GOOD	56
MINT	56

Report the following program characteristics (Hint: See the build output window in Keil).

	Size [bytes]
Program Size	9300
Read Only data	204
Read Write data	168
Zero Initialized data	512

And provide a brief explanation about which directives can influence the previous program characteristics.

The Program Size (9300 bytes) includes executable instructions and is significantly increased by `SPACE 4096` directives in the code section, plus `LTORG` for literal pools and `ALIGN` for memory alignment padding.

The Read Only Data (204 bytes) is defined by `DCD` (4 bytes each) and `DCB` (1 byte) directives that create constant tables.

The Read Write Data (168 bytes) represents initialized writable memory defined by the `AREA` directive with `DATA`, `READWRITE` attributes.

The Zero Initialized Data (512 bytes) comes from `SPACE` directives within the `READWRITE` area.

`POOR`, `GOOD`, and `MINT` arrays use `SPACE 56` each (168 bytes total), with the remaining 344 bytes likely from `SPACE 4096` directives and runtime stack/heap allocations.